

OPTIMIZATION METHODS FOR LOGISTIC REGRESSION IN SPOTIFY SONGS' POPULARITY CLASSIFICATION PROBLEM

James Ho, Aaron Amella, Paul Duft
Colorado State University



Problem Statement

The goal of this project is to explore the use of the optimization model, logistic regression, to predict whether a song will be popular with a Spotify data set. This classification problem is solved by training the model with several optimization algorithms, such as Limited-memory BFGS, gradient descent, and Newton's method, while also comparing their convergence behavior and performance. The Spotify data set consists of key audio and descriptive features, which are used as inputs to direct the output. By training a predictive classification model, we assess how effective the optimization algorithms are in minimizing the loss function, accuracy of predicting popularity, and overall efficiency.

Spotify Dataset

Source: Kaggle

Description: The data set used to study is titled "Spotify Music Dataset" and is split up into two collections: high popularity songs and low popularity songs. Each song is broken down into the following audio and descriptive features:

- Danceability
- Energy
- Loudness
- Tempo
- liveness
- Acousticness
- Speechiness
- Instrumentalness
- Valence
- Mode
- Key
- time signature
- duration

The two collections of data sets of high popularity and low popularity are used to create the binary popularity labels:

- 1 = Popular song
- 0 = Not popular song

Model Description

This is a classic binary classification problem with unpopular songs classified as 0 and popular songs classified as 1.

- input vector $x \rightarrow$ features of the song, i.e. tempo, energy, danceability, etc. that Spotify uses to categorizes music. $x \in R^n$ where n = number of features used.
- output vector $y = \begin{cases} 1 & \text{song is popular} \\ 0 & \text{song is unpopular} \end{cases}$
- input/output pairs: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ where each x_i is a feature vector and $y_i \in \{0, 1\}$

Model Algorithms

Logistic Regression:

$$\hat{y} = \sigma(w^T x + b)$$

x = song features

w = weight vector (how important each feature is)

b = bias

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \text{sigmoid function: outputs a probability between 0 and 1.}$$

Error Loss Function:

$$E(w) = \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Minimizing $E(w)$ using

BFGS:

$$\text{update rule: } w_{k+1} = w_k - \alpha_k B_k^{-1} \nabla E(w_k)$$

Gradient Descent:

$$\text{update rule: } w_{k+1} = w_k - \alpha \nabla E(w_k)$$

Newton's Method:

$$\text{update rule: } w_{k+1} = w_k - H^{-1}(w_k) \nabla E(w_k)$$

$$\nabla E(w) = \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

Used by BFGS, Gradient Descent, and Newton's method

Results

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

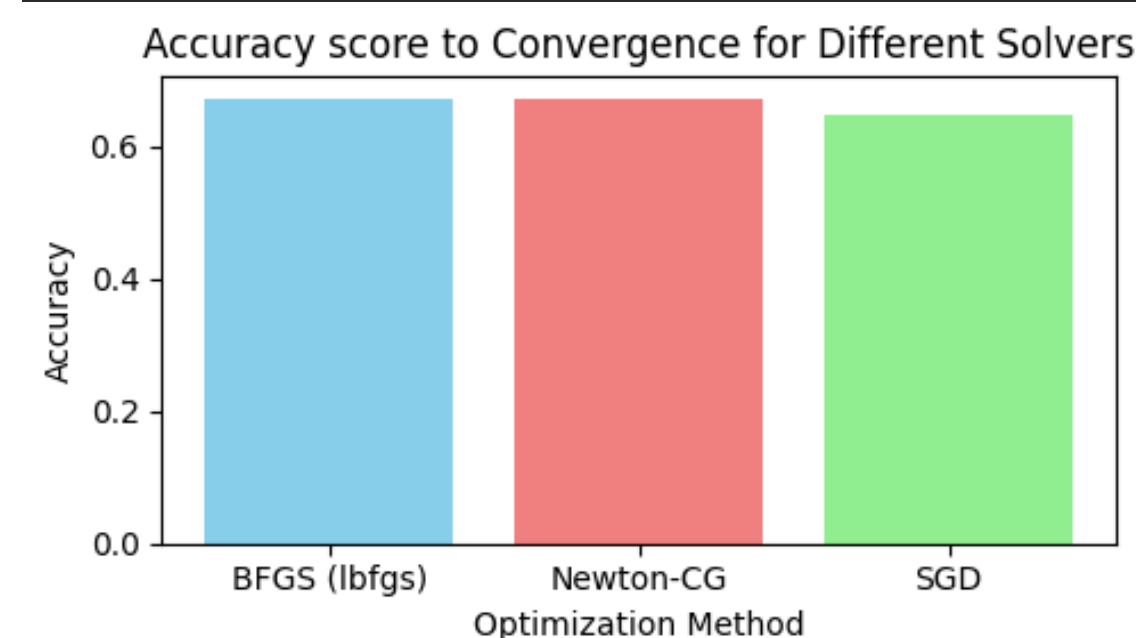
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss

model = LogisticRegression(
    penalty=None,
    solver='lbfgs', # Limited memory BFGS
    max_iter=1000
)

model.fit(X_train, y_train)
```

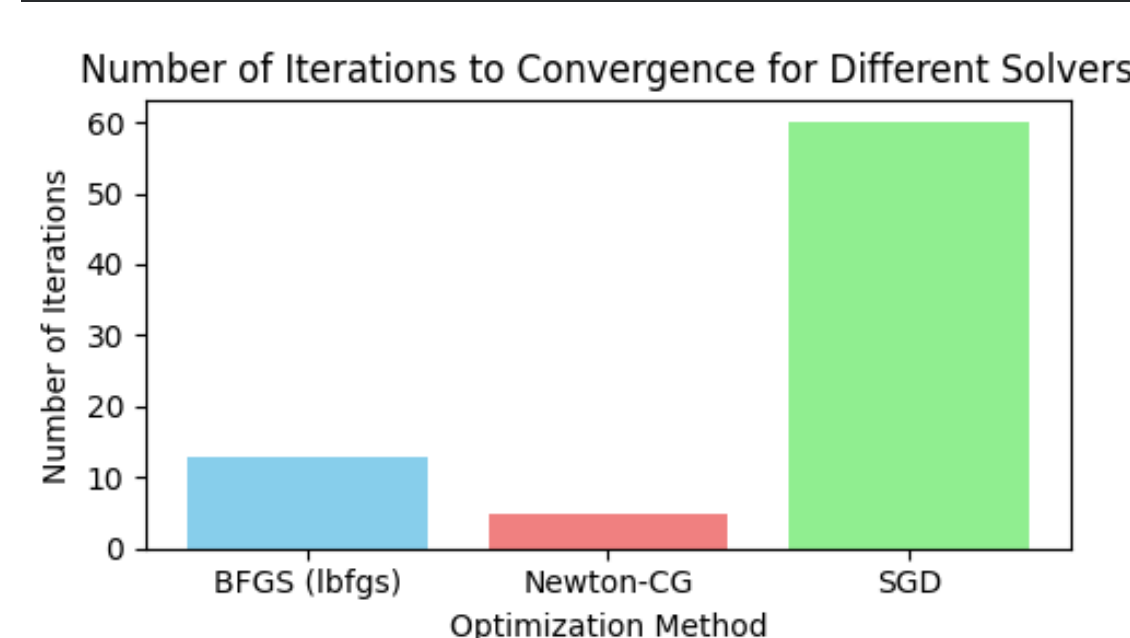


```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import mean_absolute_error

y_pred = model.predict(X_test)
prob_bfg = model.predict_proba(X_test)
y_prob_bfgs = model.predict_proba(X_test)[:, 1]
fpr_bfgs, tpr_bfgs, _ = roc_curve(y_test, y_prob_bfgs)

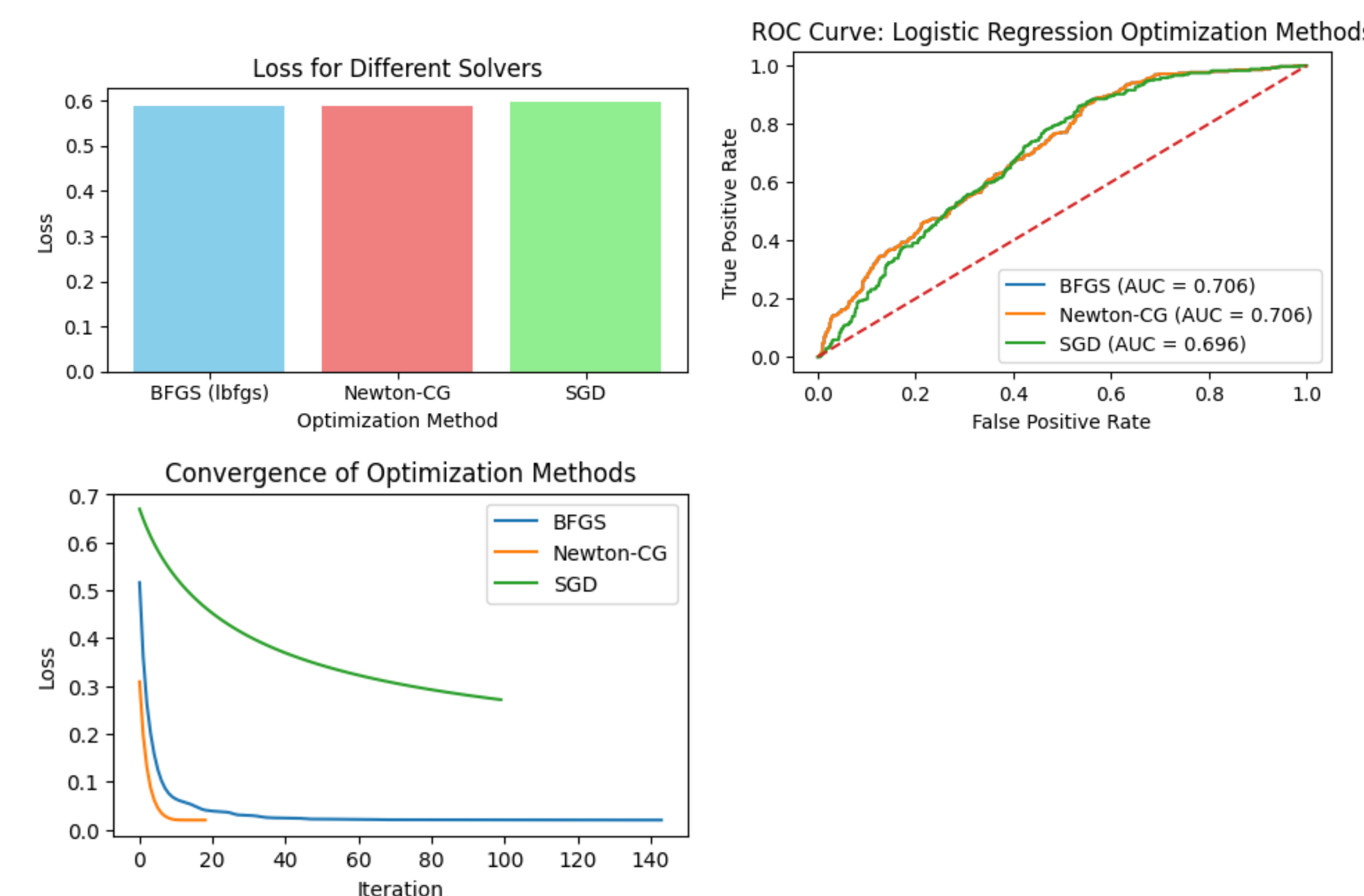
bfg_acc = accuracy_score(y_test, y_pred)
lbfgs_n_iter = model.n_iter_[0] # Store the number of iterations
bfg_loss = log_loss(y_test, prob_bfg)
bfg_mse = mean_absolute_error(y_test, y_pred)
auc_bfgs = auc(fpr_bfgs, tpr_bfgs)

print('Number of iterations:', model.n_iter_)
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
print(f'Logistic Loss: (bfg_loss)')
print(f'Mean Absolute Error: (bfg_mse)')
```



ROC Curve: Logistic Regression Optimization Methods

Results



BFGS: Number of iterations: 13, Accuracy: 0.671, Logistic Loss: 0.587

Gradient Descent: Number of iterations: 60, Accuracy: 0.646, Logistic Loss: 0.598

Newton's Method: Number of iterations: 5, Accuracy: 0.670, Logistic Loss: 0.587

Discussion

Newton's method converges in very few iterations, often achieving near-optimal solutions in under ten steps when the Hessian is well-conditioned. BFGS converges slightly more slowly in iteration count but remains competitive while avoiding the cost of Hessian inversion. Both methods significantly outperform first-order gradient descent in convergence speed.

The Spotify popularity classification task provides a clean setting to compare second-order optimization methods on a convex objective. Newton's method benefits from exact curvature information, leading to rapid convergence, but its computational cost scales poorly with feature dimension. BFGS provides a favorable trade-off by approximating curvature using gradient information alone, making it more practical for higher-dimensional problems. From a data science standpoint, both methods yield similar classification accuracy, suggesting that optimization choice primarily impacts efficiency rather than predictive performance.

Conclusion

All three optimization methods converged to a similar accuracy and logistic loss, but differed significantly in convergence speed and number of iterations. The choice of method primarily affects model training efficiency and does not largely impact accuracy.

For convex classification problems with the use of logistic regression, BFGS provides an effective balance of optimization efficiency and convergence speed. This makes BFGS a practical choice for data science applications for larger-scale problems.

Citations

Ahmed, S. (2024). *Spotify Music Dataset (Version 1)* [Data set]. Kaggle. <https://www.kaggle.com/datasets/solomonameh/spotify-music-dataset>